

AZURE DEVOPS ENVIRONMENT
SETUP.AIM:

To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

INSTALLATION:

1. Open your web browser and go to the Azure website:
<https://azure.microsoft.com/en-us/get-started/azure-portal>
Signin using your Microsoft account credentials. If you don't have a Microsoft account, you can create one here:
<https://signup.live.com/?lie=1>
2. In Azure home page, open DevOps environment in azure platform by typing Azure DevOps organizations in the search bar
3. Click on the My Azure DevOps organization link and create an organization and you should be taken to the Azure DevOps organization Home Page.

RESULT:

Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT

AIM:

To establish an Azure DevOps project environment that enables efficient team collaboration, streamlined development workflows and effective agile project management through structured planning, tracking and execution of tasks.

PROCEDURE:

1. Creating an Azure Account:

The first step in setting up an Azure DevOps environment is to create a Microsoft Azure account. This account serves as the foundation for accessing all Azure services, including Azure DevOps.

To begin, navigate to the official Microsoft Azure website and select the option to create a new account. You will be required to provide basic details such as your email address, phone number, and payment information. Microsoft may offer a free trial credits for new users, which can be utilized during initial step and examination.

After completion, verify your account through the confirmation steps provided. Once verification is complete, you can login to the Azure portal and access the Azure DevOps services.

2. Creating an Organization in Azure DevOps

After successfully creating an Azure account, the next step is to create an organization within Azure DevOps. An organization acts as a container for all projects, users, repositories and pipelines.

Upon logging into Azure DevOps, you will be prompted to create a new organization if one does not already exist. You will need to provide a unique name for your organization and select a region where your data will be hosted.

The region selection is important as it determines data residency and performance.

Once the organization is created, you will be redirected to the organization's home page, where you can begin creating and managing projects.

3. Creating the First Project in the organization.

After setting up the organization, the next step is to create a project. A project in Azure DevOps is where all development activities such as source code management, work tracking and CI/CD pipelines are organized.

To create a project:

- Navigate to organization's home page
- Click on the "New Project" button
- Enter the required project details:
 - Name: Provide a meaningful and concise name for the project
 - Description: optionally include a description that explains the purpose, scope or objectives of the project
 - Visibility: • Private • Public.

4. Accessing and Navigating the organization.

After logging into Azure DevOps, it is important to ensure that you are working within the correct organization, especially if you are part of multiple organizations.

To verify or switch organizations:

- Locate the organization name in the top-left corner of the interface
- Click on it to view a list of available organizations
- Select the desired organization to switch context.

5. understanding the project dashboard.

The project dashboard provides a comprehensive overview of the project's status and progress. It acts as a centralized interface where team members can monitor ongoing activities and access various tools.

Key components of the dashboard include:

- Overview section: Displays project summary and recent activity.
- Boards: Used for managing work items such as user stories, tasks and bugs.
- Repos: Hosts source code repositories and version control features.
- Pipelines: Enables continuous integration and deployment workflows.
- Artifacts: Manages packages and dependencies.

The dashboard can be customized by adding widgets such as charts, sprint progress and build status to provide better visibility and project performance.

6. Managing user stories using agile boards.

User stories are a fundamental component of agile methodology. They represent features or requirements from the end-user's perspective and help teams plan and prioritize work effectively.

To manage user stories in Azure DevOps:

- From the left-hand navigation menu, select "Boards".
- This will open the main boards interface, where you can view backlogs, sprints, and work items.

To create a new user story:

- Locate the option labeled "Add a work item" at the top of the page.

- Alternatively, use the "+" icon or 'Add New work Item' option depending on the current view.

- From the dropdown menu, select "User story".

A form will appear where you can enter detailed information including:

- Title: A concise description of the user requirement
- Description: A detailed explanation of the feature or functionality.
- Acceptance criteria: Conditions that must be met for the story to be considered complete.
- Priority: Defines the importance of the user story.
- Assigned To: Specifies the team member responsible for the task.

7. Agile workflow setup:

Azure DevOps supports agile methodologies by providing tools such as Kanban boards, sprint planning and backlog management.

Teams can:

- organize work into iterations (sprints)
- Prioritize tasks using backlogs
- Track progress visually using Kanban boards
- Monitor team performance through analytics and reports.

RESULT:

Successfully created and configured an Azure DevOps project environment with a structured organization, project setup and agile workflow integration.

DATE:
29/01/26AIM:

To learn about creating and to organize Epics, Features, User stories, and Tasks in Azure DevOps for the to-do List Management system.

Project Description:

The project is a Task Management System that allows users to manage tasks efficiently. The application includes user authentication, mode selection, task management, and a reminder system for due dates.

I. Epics Created:

1. User Authentication System
2. Mode Selection System
3. Personal Task Management
4. Professional Team Management
5. Reminder and Notification system.

II. Features Created:

1. User sign-up and login
2. Mode Selection (Personal / Professional)
3. Create, Edit, Delete Personal Tasks
4. Create Team Group and Add Members
5. Assign Tasks to Team Members
6. Task Reminder and Notification system.

III. User stories Details:

1. As a user, I want to create an account so that I can use the application
2. As a user I want to login so that I can access my tasks.
3. As a user, I want to choose between personal and professional mode.

4. As a user, I want to create tasks with description and due date.

5. As a user, I want to edit or delete tasks in personal mode

6. As a team leader, I want to create a group and add members.

7. As a team leader, I want to assign tasks to team members.

8. As a user, I want to receive reminders one week before the due date.

14. Tasks created

- Design login and signup UI
- Implement authentication logic
- Develop mode selection interface
- Create task input form with fields
- Implement task creation, editing and deletion
- Develop team group and member management
- Implement task assignment functionality.
- Develop reminder system for due dates.
- Test all modules of the application.

RESULT:

Thus, the creation of Epics, Features, User stories and Tasks for the ~~To-do list~~ Management system has been successfully completed using Azure Devops.

DATE:
05/02/26AIM:

To assign user stories to specific sprints for the To-Do List management system Project using Azure DevOps.

SPRINT PLANNING:Sprint 1: Authentication & setup

- User sign-up functionality
- User Login Functionality
- Basic UI design for login and signup
- Validation of user credentials.

Sprint 2: Mode selection & Personal Tasks

- Mode selection (Personal / Professional)
- Create Personal Tasks.
- Edit Personal Tasks.
- Delete Personal Tasks.

Sprint 3: Professional Team Management

- Create Team Group
- Add Team Members
- Assign Tasks to Team Members
- Display Tasks for team users.

Sprint 4: Reminder System & Testing

- Implement due date tracking
- Develop reminder notification
- Integrate all modules
- Testing and debugging of the application.

RESULT:

The sprints have been successfully created and user stories have been assigned for the task management system project using Azure DevOps.

AIM:

To create Pocker Estimation for the user stories of the Task Management System Project using Azure DevOps.

POKER ESTIMATION

Poker Estimation is used to assign story points based on complexity, effort and time required for each user story.

USER STORY ESTIMATION:

- User Story sign-up → 3 story Points.
- User Login → 3 story Points.
- Mode Selection (Personal / Professional) → 2 story Points
- Create Personal Task → 5 story Points.
- Edit Personal Task → 3 story points.
- Delete Personal Task → 3 story points.
- Create Team Group → 5 story points
- Add Team Members → 5 story points
- Assign Tasks to Team Members → 8 story Points
- Reminder Notification System → 8 story Points
- Testing and Integration → 5 story points.

ESTIMATION CRITERIA :

- 1-2 points → Simple
- 3-5 points → Moderate
- 8 points → complex.

RESULT:

The story points for all user stories have been successfully assigned using Pocker Estimation for the Task Management System project.

EX NO: 6

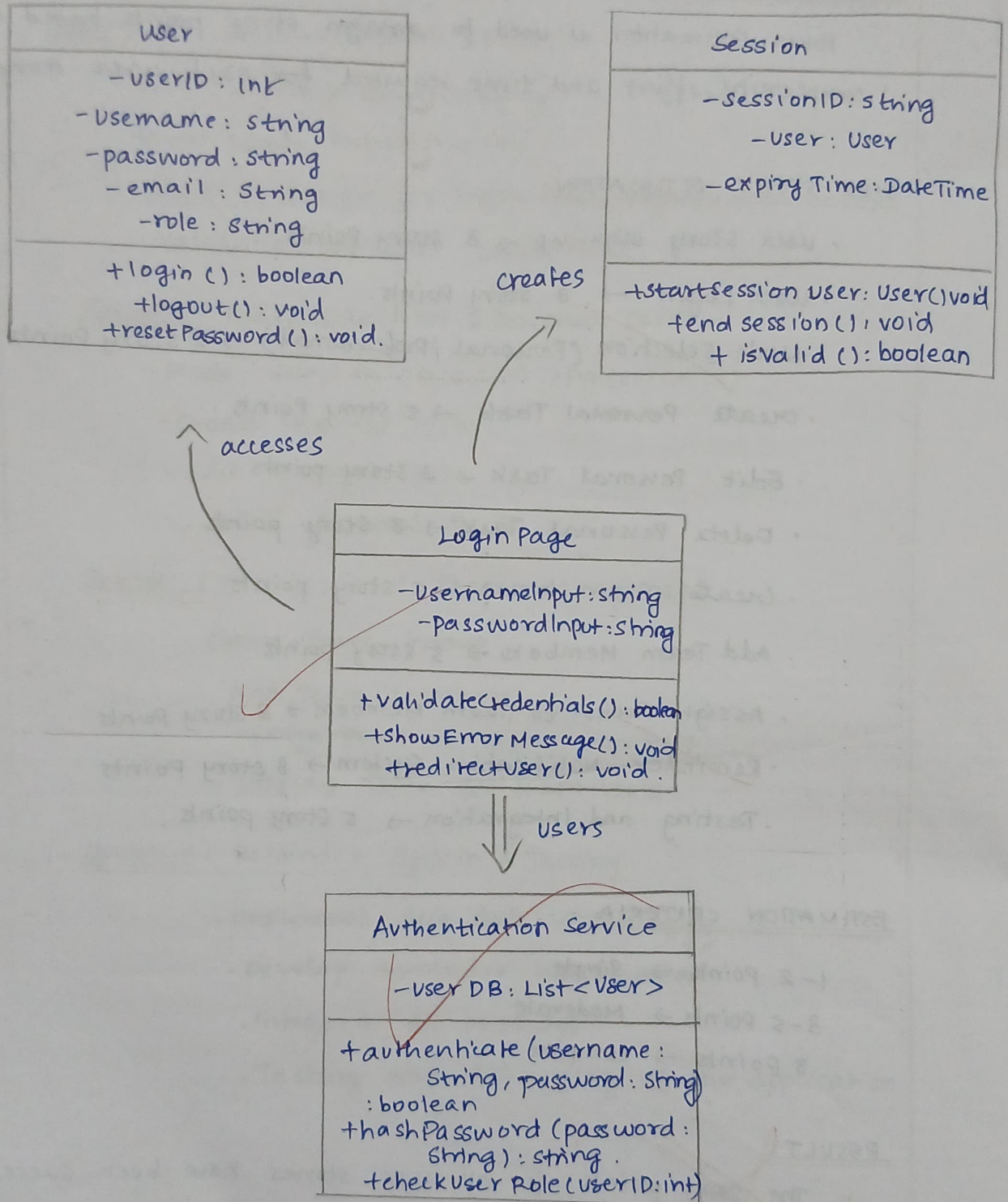
DATE:
19/02/26

DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PROJECT ARCHITECTURE.

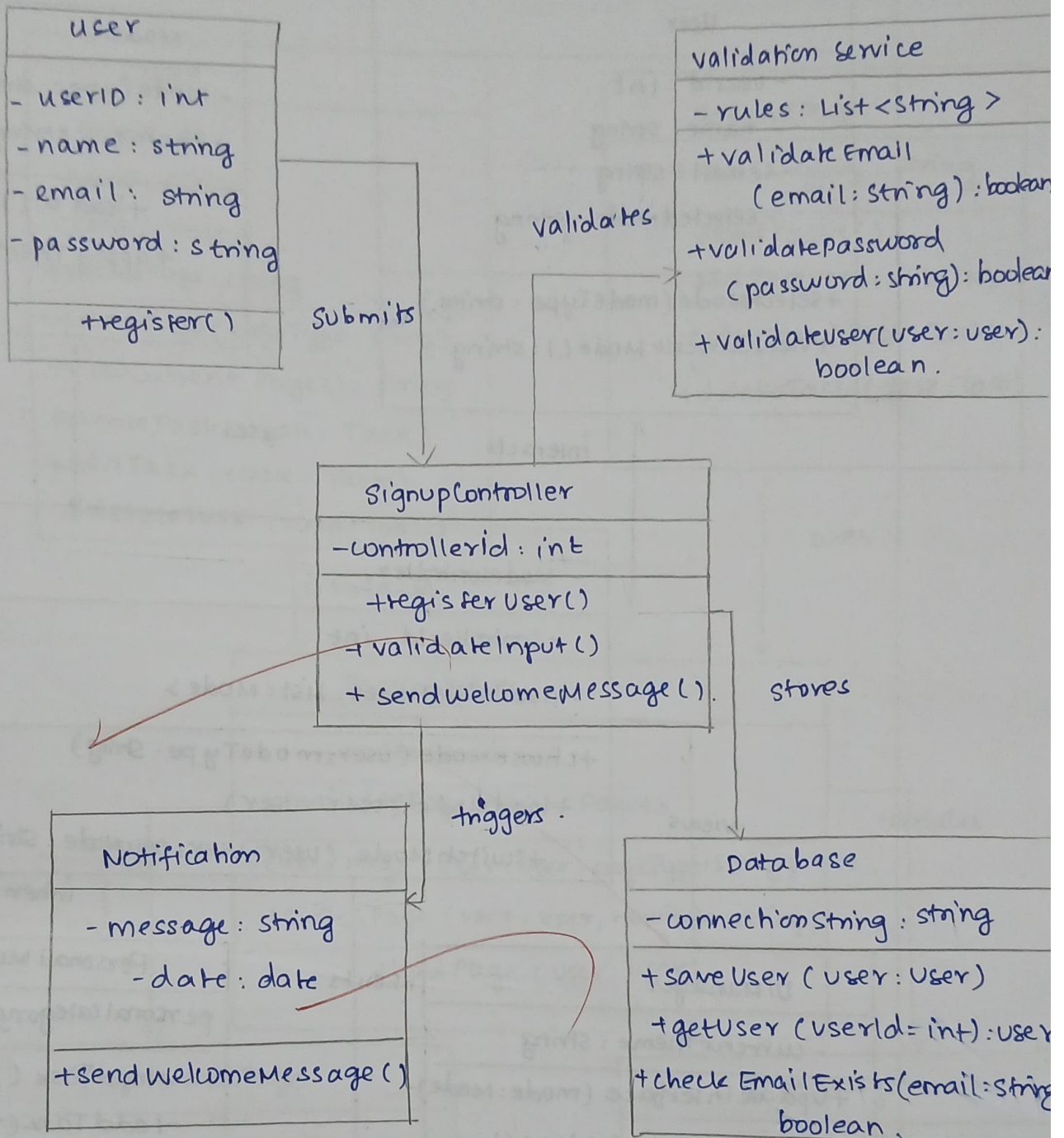
AIM:

To design a class diagram and sequence diagram for a task management system.

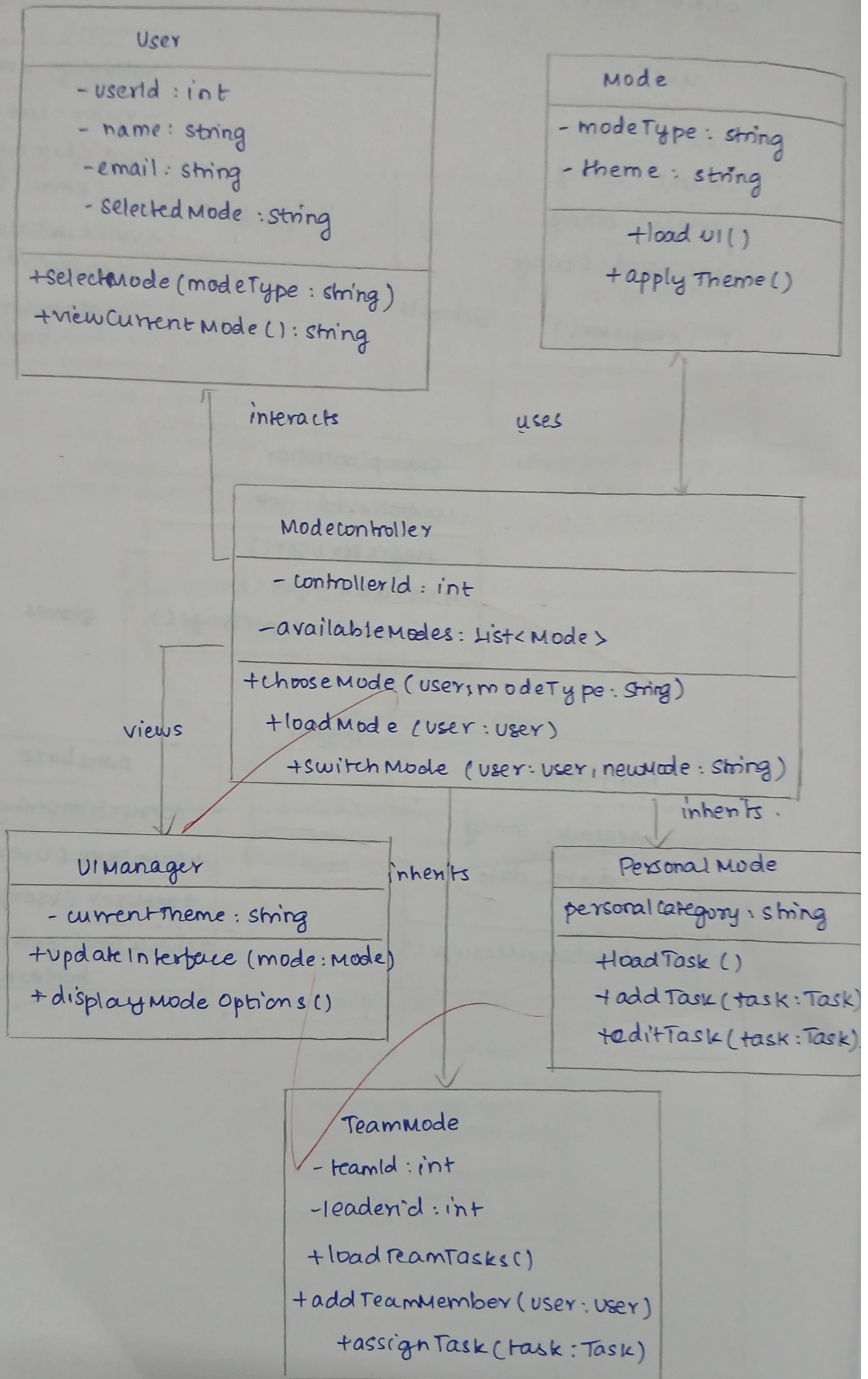
LOGIN PAGE:



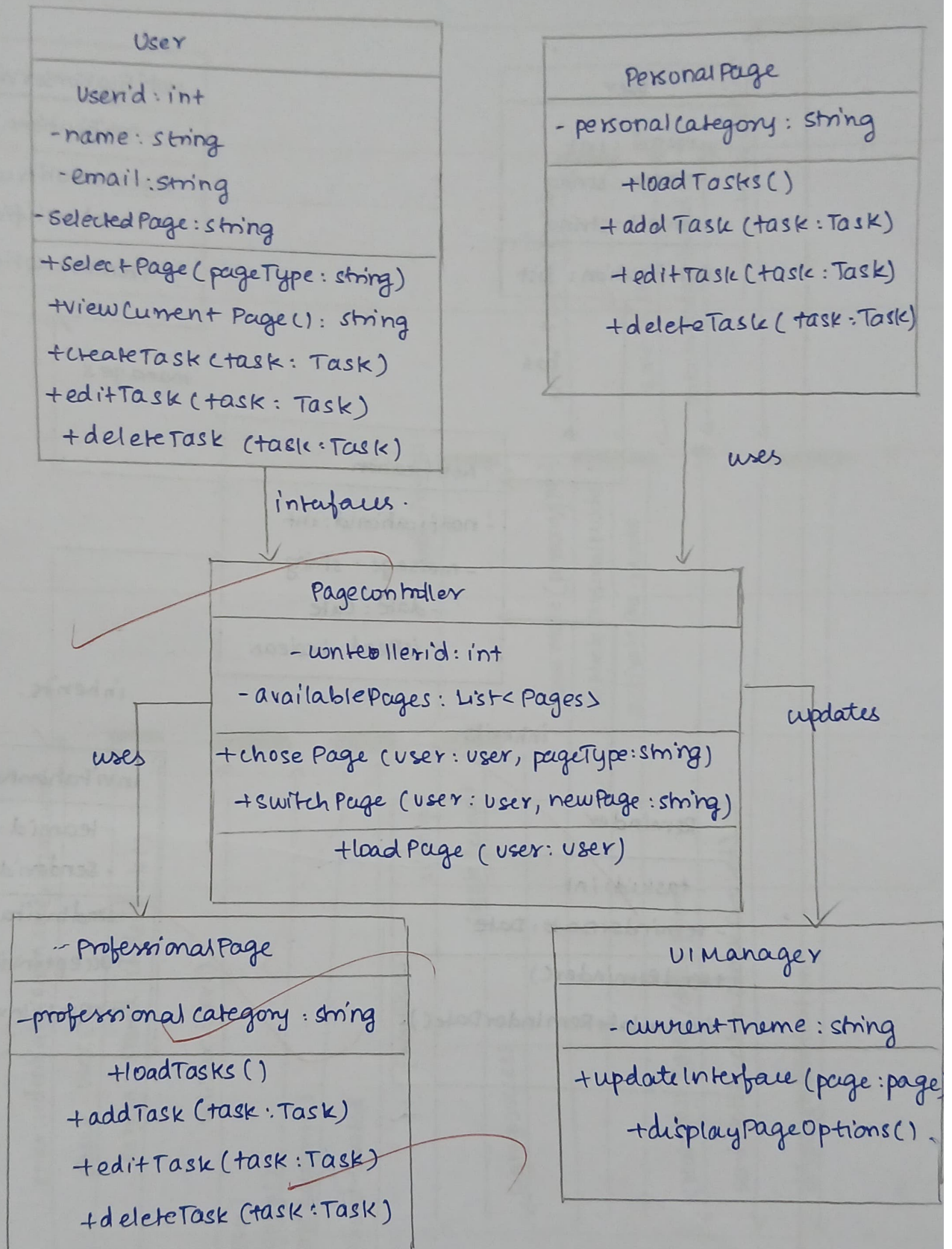
SIGNUP PAGE:



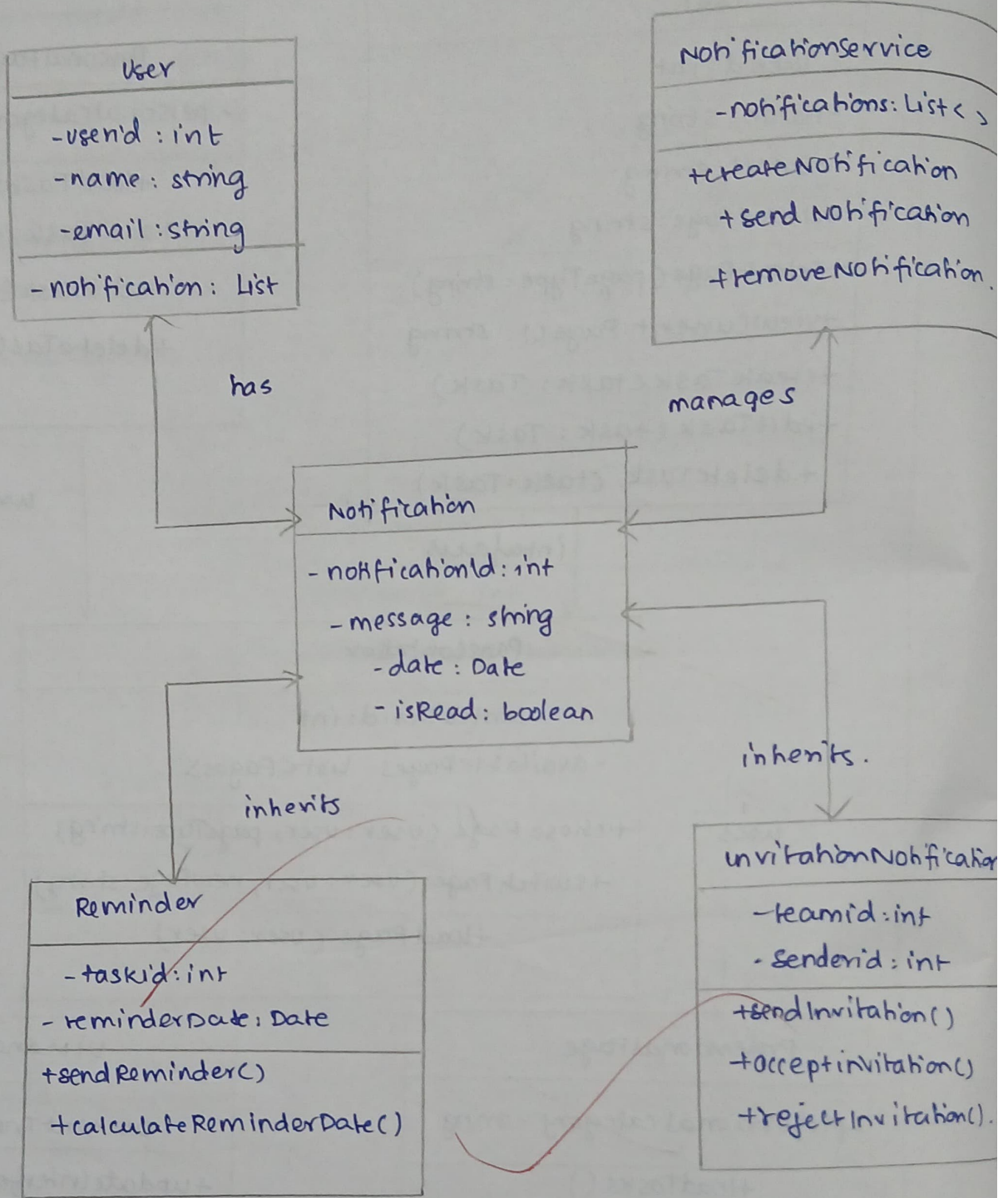
MODE CONTROLLER:



PRO/PERSONAL MODE PAGE:



NOTIFICATION SYSTEM PAGE.



SEQUENCE DIAGRAM:

Db

Notification

Professional

Personal

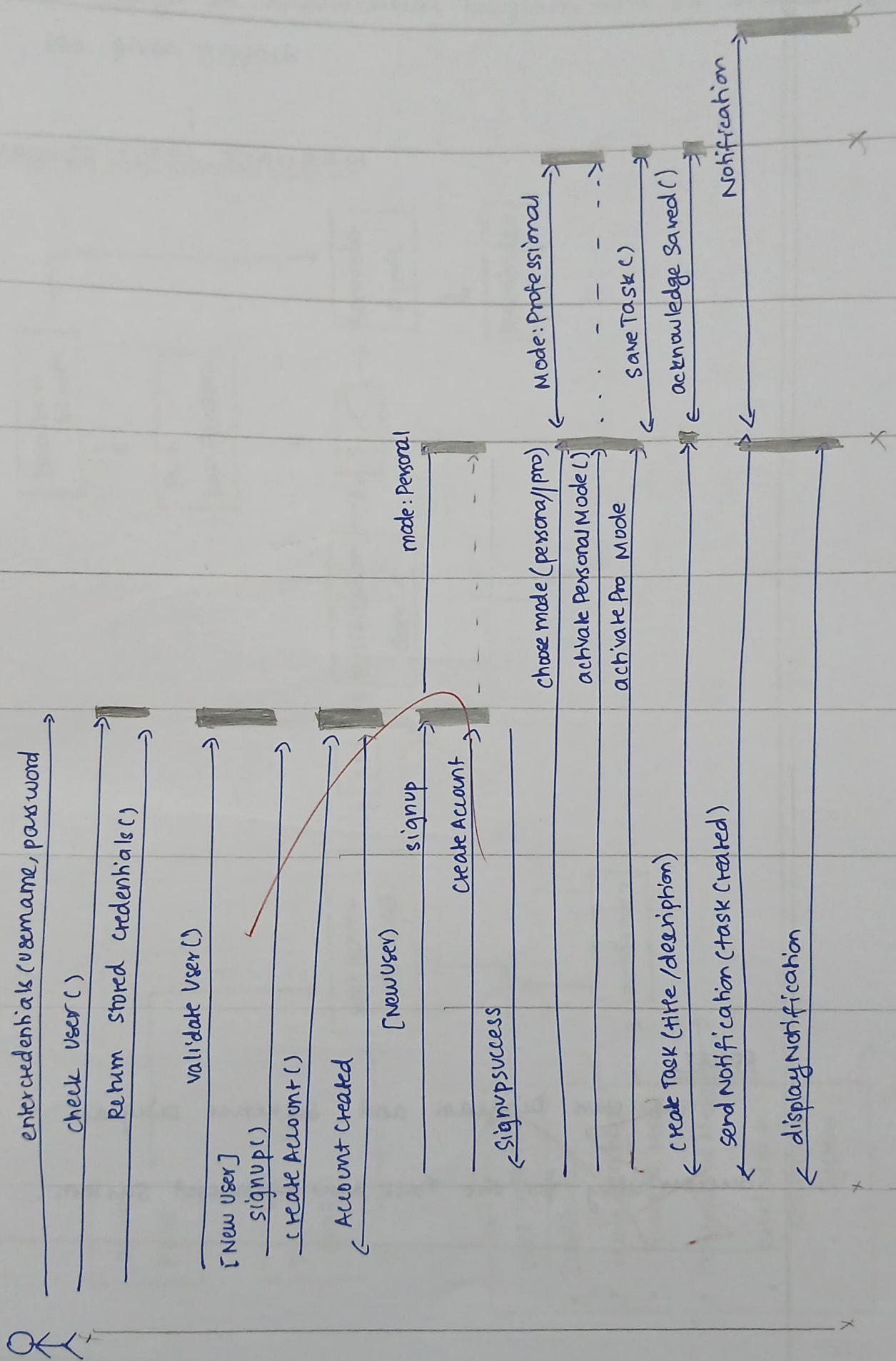
Mode

Reminder

Signup

LoginPage

User



RESULT:

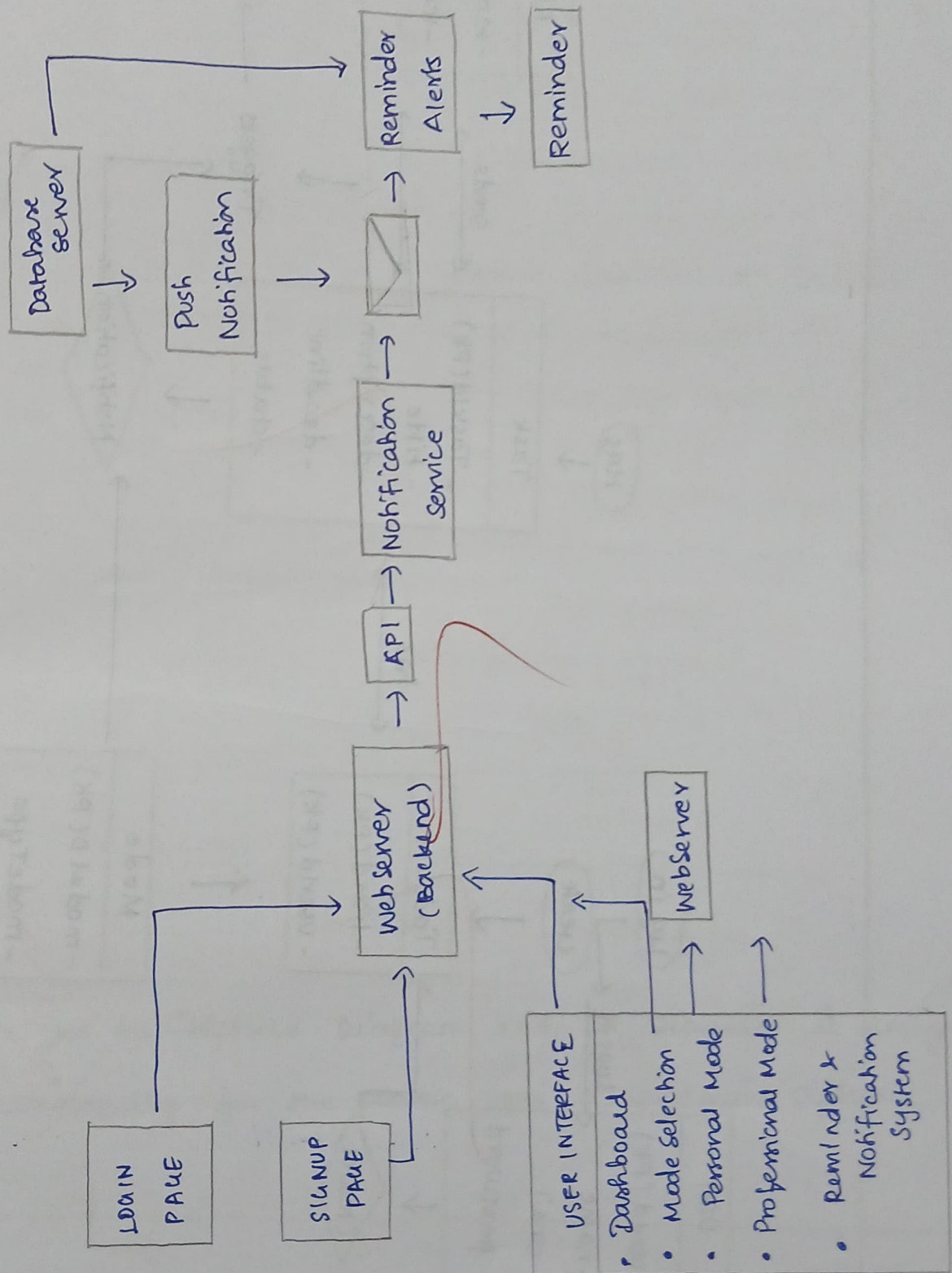
The class Diagram and Sequence Diagram is designed successfully for the Task Management System.

DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PROJECT STRUCTURE.

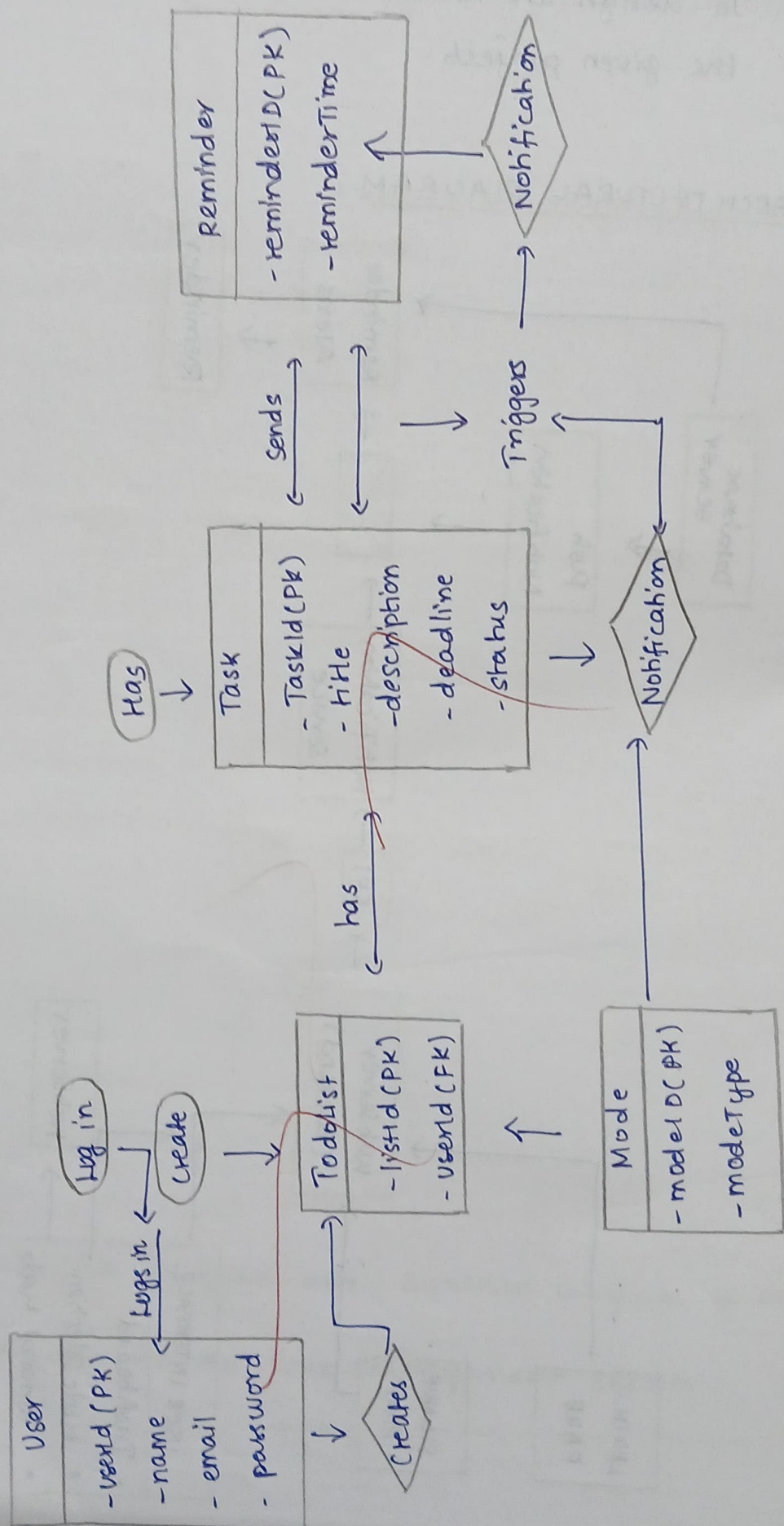
AIM:

To design an Architectural diagram and ER Diagram for the given project.

ARCHITECTURAL DIAGRAM:



ER DIAGRAM:



RESULT

The Architecture Diagram and ER Diagram is designed Successfully for the ~~Task~~ Task Management System.

EX NO: 8

DATE:

12-3-26.

TESTING - TEST PLANS AND TEST CASES .

AIM:

To create Test Plans and Test cases for the Task Management System and write test cases for user stories covering both Happy path and Error scenarios using Azure DevOps .

TEST CASE DESIGN PROCEDURE:

Understand core Features:

- User Signup & Login
- Mode Selection
- Personal Task Management
- Professional Team Task Management
- Reminder Notification System.

Define user interaction:

Each test case simulates real user behavior .

- Signing up
- Logging in
- Adding tasks
- Managing team tasks

Design Happy path test cases

- Valid inputs
- Successful operations

Design Error Path Test cases:

- Invalid inputs
- System failures
- Edge cases

Break Down Steps & Expected Results

Each test case includes:

- Actions
- Expected outcomes.

Use clear Naming & IDs:

TC01 - Successful Login

TC02 - Invalid Login

Test Suites organization

- Login
- Task Management
- Team Features
- Reminder System

1. New Test Plan

Test Plan Name:

Task Management System - Test Plan

2. Test suites & Test Cases:

Test suite: TS01 - User Authentication

User story 1

As a user, I want to sign up and log in securely so that I can access my tasks.

TC01 - Successful sign up (Happy Path)

Action:

Go to signup page

Enter valid username & password

click "signup"

Expected Result:

Account created

Redirected to login page

TC02 - signup with existing username (Error Path)

Action:

Enter already registered username

click sign up

Expected Result:

Error: "user already exists"

TC03 - Successful Login (Happy Path)

Action:

Enter valid credentials

click Login

Expected Result:

Redirect to dashboard.

TC04 - Invalid Login (Error Path)

Action:

Enter wrong password

click Login

Expected Result:

Error: "Invalid credentials"

Test suite: TS02 - Mode Selection.

User story 2

As a user, I want to choose between personal and professional modes.

TC05 - Select Personal mode (Happy Path)

Action:

Login

Click "Personal"

Expected Result:

Redirect to personal dashboard

TC06 - Mode selection Failure (Error Path)

Action:

Click mode without login

Expected Result:

Redirect to login page

Test Suite: TS03 - Personal Task management

User story 3

As a user, I want to add, edit and delete personal tasks

TC07 - Add task successfully (Happy Path)

Action:

Enter Task details

Click "Add Task"

Expected Result:

Task added to list.

TC08 - Add task with missing field (Error Path)

Action:

leave fields empty

click Add

Expected Result:

Error: "All fields required"

TC09 - Edit Task (Happy Path)

Action:

Modify Task

Save changes

Expected Result:

Task updated

TC10 - Delete Task (Happy Path)

Action:

Click delete

Expected Result:

Task Removed.

Test suite: TS04 - Professional Team Management
User story 4.

As a team leader, I want to manage team members
and assign tasks.

TC11 - Create Team (Happy Path)

Action:

Enter team name

Click create

Expected result:

Team created

Test suite : TS05 - Reminder system

User story 5

As a user, I want reminders before due dates

TC15 - Reminder Trigger (Happy Path)

Action:

create task with due date

Expected Result:

Reminder shown before a week.

3. Installation of test

Install Azure test plans extension

connect to project

4. Running Test cases

Select test suites

execute test cases.

5. Recording Test cases

Record execution steps

capture screenshots

6. Creating bugs

log bugs if test fails

Assign bugs to developer.

7. Test Results

Passed / Failed status

Track defects

8. Test Report summary

Total test cases

passed / failed count

9. Progress report

Track testing progress

Monitor bug fixes.

10. Modify test templates

customize test fields.

11. View test templates

Review updated templates.

RESULT :

Test plans and test cases are made for the Test management System.

LOAD TESTING AND PIPELINES.

AIM:

To create an Azure Load Testing resource and run a load test to evaluate the performance of a Task Management System API endpoint and to create and demonstrate an Azure DevOps pipeline for automating application builds, tests and deployment.

LOAD TESTING:

Azure Load Testing is used to simulate multiple users accessing the Task Management system to evaluate system performance under stress. It helps identify bottlenecks, latency issues.

STEPS TO CREATE AN AZURE LOAD TESTING RESOURCE:

1. Sign in to Azure Portal

Go to <https://portal.azure.com> and log in

2. Create the resource

Navigate to create a resource

Search for Azure Load Testing.

click create.

3. Fill in configuration details.

Subscription: select your Azure subscription.

Resource group: create new or use existing.

Name: provide a unique name.

Location: choose region.

4. click review + create → create.

5. After Deployment, click go to resource.

Steps to create and run a load test:

1. open your Azure load testing resource.
2. click Add Http request → create.

Basics Tab:

Test Name: Task Management - LoadTest

Description: Performance testing of Task Management API.
Keep run after creation enabled.

3. click Review + create → create.

Expected outcome of load testing.

- Response time for task operations
- Throughput
- Error rate under high traffic.
- Identification of slow endpoints.

Pipeline:

Description:

This experiment demonstrates integrating a Flask-based Task Management System with Azure DevOps. The pipeline automates dependency installation, environment setup and basic validation checks to ensure reliability.

Steps:

1. connect Github to Azure DevOps
 - create a new project in Azure DevOps
 - create a pipeline
 - select Github as source
 - Authorize repository access.

pipeline tasks include :

setting up python environment

installing flask project dependencies .

verifying application environment setup .

ensuring pipeline executes successfully on each commit .

Run and monitor pipeline .

Push changes to main branch .

Pipeline triggers automatically

Monitor logs in Azure DevOps dashboard .

verify successful execution or debug errors .

RESULT:

Successfully created the Azure Load Testing resource .
and executed a load test to evaluate the performance
of the Task management system endpoints .

GITHUB: PROJECTS STRUCTURE. & NAMING CONVENTION.

AIM:

To provide a clear and organized view of the projects folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Task Management System project.

GITHUB PROJECT STRUCTURE:

Task-Management-System/

— project/

— static/

— templates/

— app.py

— models.py

— routes.py

— database.db

— requirements.txt

— tests/

— .gitignore

— README.md

— azure-pipelines.yml

— LICENCE.

FILE NAMING CONVENTIONS:

- Use lowercase with underscores
- Keep names descriptive and meaningful
- Separate concerns:
 - routes.py → API logic
 - models.py → Database schema
 - templates/ → UI files.

RESULT:

The Github repository clearly presents a well-structured Task management system with consistent naming conventions, enabling easy navigation, understanding and future development by contributors.